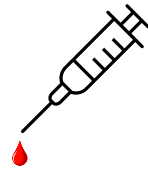
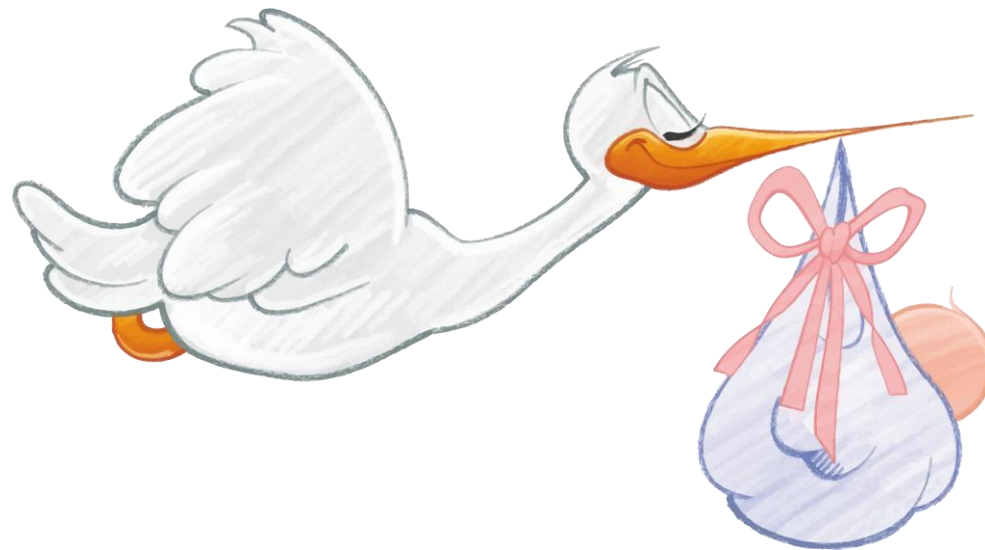


Dependency Injection



Wo kommen eigentlich die Objekte her?

new Baby();



new: wo und wann?

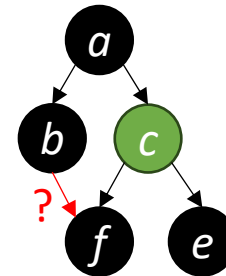
Grundproblem: Wo und wann sollen Objekte in einer Software erzeugt werden?

🤔 **Idee 1:** Immer in der Klasse, wo sie benötigt werden.

🤔 **Problem 1:**

c erzeugt f und e .

Wenn b auch auf dasselbe f zugreifen soll,
ist doof, weil b nicht einfach an f herankommt.



```
class C {  
    private F f = new F();  
    private E e = new E();  
  
    // do something with f and e...  
}
```

```
class B {  
    // How to get object f in B??  
    private F f;  
  
    // do something with f...  
}
```

🤔 **Problem 2:**

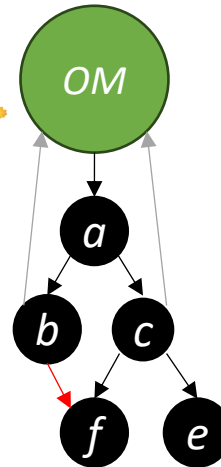
Wenn für c ein Unit Test geschrieben wird, müssen f und e mitgetestet werden,
da diese fest verdrahtete Abhängigkeiten von c sind.

new: wo und wann?

Grundproblem: Wo und wann sollen Objekte in einer Software erzeugt werden?



Idee 2: Zentraler "Objekt-Manager" (*OM*).



OM erzeugt *a*, *b*, *c*, *f* und *e*.

Zugriff auf die Objekte erfolgt über statische Getter in *OM*.



Probleme:

Die Objekte von *OM* lassen sich für Unit-Tests nicht leicht austauschen (erfordert z. B. Hilfswerkzeuge wie Mockito).

Man könnte statische Setter in *OM* hinzufügen, aber das würde zu ungewollten Seiteneffekten führen, da die Ausführungsreihenfolge der Setter dann wichtig wäre.

```
class OM {  
    private static A a = new A();  
    // ...  
    private static E e = new E();  
    public static getA() {return a;}  
    // ...  
    public static getE() {return e;}  
}
```

```
class C {  
    private F f = OM.getF();  
    // do something with f...  
}
```

```
class B {  
    private F f = OM.getF();  
    // do something with f...  
}
```

new: wo und wann?

Grundproblem: Wo und wann sollen Objekte in einer Software erzeugt werden?

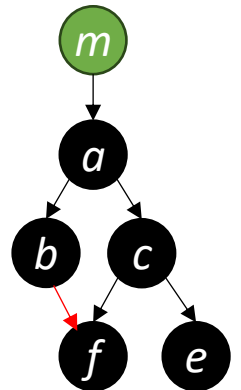
🤔 **Idee 3:** Ganz "oben" in der main-Methode (*m*).

m erstellt *a*, *b*, *c*, *f* und *e*.

m übergibt *b* und *c* an *a* sowie *f* und *e* an *c* (per Konstruktor).

Falls *b* auch *f* benötigt, kann *m* einfach *f* an *b* per Konstruktor mit übergeben.

Für Unit Tests lassen sich Klassen über den Konstruktor austauschen! 😊



🤔 **Problem:**

Bei großen Projekten mit vielen Klassen und unterschiedlichen Konstruktoren wird das schnell anstrengend wegen der Erzeugungsreihenfolge.

```
public static void main(String[] args) {  
    F f = new F();  
    E e = new E();  
    C c = new C(f, e);  
    B b = new B();  
    // new B(f);  
    A a = new A(b, c);  
}
```

Dependency Manager

Kann man die lästige Verwaltung von Objekten nicht delegieren?

Ja, es gibt Dependency Manager Frameworks, die genau dies machen.

Ein bekanntes Beispiel für Java ist z. B. [Spring](#). 

Dependency Injection

Abhängigkeiten über einen Konstruktor (oder Setter) zu übergeben, nennt man **Dependency Injection**.

```
interface TranslationService {  
    String translate(String text);  
}
```

```
class Translator { // generischer Sprachen-Übersetzer
```

```
    private TranslationService translationService;
```

Konstruktor

```
    Translator(TranslationService translationService) {  
        this.translationService = translationService; // speichere das übergebene Objekt für später im Attribut ab.  
    }  
}
```

```
String translate(String text) {  
    return translationService.translate(text);  
}
```

```
void main() {
```

```
    var deEnTranslator = new Translator(new GermanEnglishTranslator()); // Erzeuge alle Objekte und erfülle ihre Abhängigkeiten mittels Dependency Injection.  
    IO.println(deEnTranslator.translate("Hallo")); // returns Hello
```

```
    var deKlTranslator = new Translator(new GermanKlingonTranslator()); // Erzeuge alle Objekte und erfülle ihre Abhängigkeiten mittels Dependency Injection.  
    IO.println(deKlTranslator.translate("Hallo")); // returns Kapla
```

```
}
```

"Die Klasse Translator hängt von der Klasse TranslationService ab."
Das benötigte Objekt (die Abhängigkeit) wird hier über den Konstruktor injiziert z. B. von der main-Methode oder von einem Testfall (Unit Test).

Hier wird die injizierte Abhängigkeit vom Translator aufgerufen.